

Edge cases (empty vs. missing) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the strict, direct way

```
# env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError("Required environment variable is missing: " + key)
    return value

# case 1: simulate an empty .env file -- the key was never set
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print("Caught it:", e)

# case 2: simulate .env containing "OPENAI_API_KEY=" -- present, but empty
os.environ["OPENAI_API_KEY"] = ""
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print("Caught it:", e)
```

Expected output:

```
Caught it: Required environment variable is missing: OPENAI_API_KEY  
Caught it: Required environment variable is missing: OPENAI_API_KEY
```

Both edge cases raise the same error here — that's the strict choice, and it's the right default for a secret like an API key. This version works correctly. It's missing type hints, which is fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — strict: empty string counts as missing

```
# env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

# case 1: key was never set at all
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 1 - not set at all: {e}")

# case 2: key is set, but empty
os.environ["OPENAI_API_KEY"] = ""
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 2 - set but empty: {e}")
```

Expected output:

```
Case 1 - not set at all: Required environment variable is missing: OPENAI_API_KEY
Case 2 - set but empty: Required environment variable is missing: OPENAI_API_KEY
```

Approach 2 — lenient: only `None` counts as missing

```
# env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None:
        raise MissingConfigError(f"Required environment variable is missing: {key}")
    return value

# case 1: key was never set at all
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 1 - not set at all: {e}")

# case 2: key is set, but empty
os.environ["OPENAI_API_KEY"] = ""
result = require_env("OPENAI_API_KEY")
print(f"Case 2 - set but empty, returned: {result!r}")
```

Expected output:

```
Case 1 - not set at all: Required environment variable is missing: OPENAI_API_KEY
Case 2 - set but empty, returned: ''
```

Difference from Basic: both Intermediate approaches add full type hints (`key: str` , `-> str`), and both test the two edge cases directly with real, side-by-side code instead of describing them in words. The real difference is between the approaches themselves: Approach 1 raises in case 2, Approach 2 doesn't — it returns the empty string instead. That's a genuinely different behavior, not just a style choice, and which one is "right" depends on whether an empty value is ever a valid setting for that particular key.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — strict, with structured error data and whitespace handling

```
# env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    def __init__(self, key: str) -> None:
        self.key = key
        super().__init__(f"Required environment variable is missing: {key}")

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is not None:
        value = value.strip()
    if value is None or value == "":
        raise MissingConfigError(key)
    return value

os.environ["OPENAI_API_KEY"] = "    " # whitespace only -- not a usable key

try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Caught it: {e}")
    print(f"Missing key was: {e.key}")
```

Expected output:

```
Caught it: Required environment variable is missing: OPENAI_API_KEY
Missing key was: OPENAI_API_KEY
```

This treats a whitespace-only value as missing too (by `.strip()`-ing before the check), and the exception now carries `self.key` directly — a caller can read `e.key` to know exactly which setting failed, without parsing the message string.

Approach 2 — one configurable function instead of two separate ones

```
# env_config_practice.py – Edge cases section
import os
from typing import Optional

class MissingConfigError(Exception):
    def __init__(self, key: str) -> None:
        self.key = key
        super().__init__(f"Required environment variable is missing: {key}")

def get_env(key: str, *, allow_empty: bool = False) -> Optional[str]:
    value = os.getenv(key)
    if value is None:
        raise MissingConfigError(key)
    if value == "" and not allow_empty:
        raise MissingConfigError(key)
    return value

os.environ["OPENAI_API_KEY"] = ""

# strict mode (the default): empty string is rejected
try:
    get_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Strict mode: {e}")

# lenient mode: empty string is allowed through
result = get_env("OPENAI_API_KEY", allow_empty=True)
print(f"Lenient mode, returned: {result!r}")
```

Expected output:

```
Strict mode: Required environment variable is missing: OPENAI_API_KEY
Lenient mode, returned: ''
```

Instead of maintaining two near-identical functions (Intermediate's Approach 1 and 2), this makes strict vs. lenient a parameter on one function. Every caller in the codebase shares the same validation logic, and only opts into "empty is fine" explicitly, per call, with `allow_empty=True`.

Approach 3 — declared with Pydantic, instead of an `if` check

```
# env_config_practice.py – Edge cases section
import os
from pydantic import BaseModel, Field, ValidationError

class Config(BaseModel):
    openai_api_key: str = Field(min_length=1)

os.environ["OPENAI_API_KEY"] = ""

try:
    config = Config(openai_api_key=os.getenv("OPENAI_API_KEY") or "")
except ValidationError as e:
    error = e.errors()[0]
    print(f"Caught it: {error['msg']}")
    print(f"Field: {error['loc'][0]}")
```

Expected output:

```
Caught it: String should have at least 1 character
Field: openai_api_key
```

There is no hand-written `if` statement here at all. `Field(min_length=1)` is the rule — Pydantic checks it automatically the moment you try to build a `Config`. (Note: `os.getenv(...)` or `""` is only there to turn `None` into `""` so both edge cases hit the *same* `min_length` rule; a real config loader would usually route the "never set" case through its own check first.)

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's two approaches are each a single, fixed behavior — you commit to strict or lenient by which function you write. Approach 1 stays with a single fixed behavior (strict) but makes the exception itself more useful (`e.key`) and closes a 3rd real gap (whitespace-only values). Approach 2 turns strict-vs-lenient into a *choice made at the call site* (`allow_empty=True` or not), so one function serves both needs instead of two separate ones. Approach 3 is the most different in kind: it doesn't write validation logic at all — it declares the rule as data (`Field(min_length=1)`) and lets a library enforce it, which is how many real production codebases handle config once they have more than a handful of fields to validate.

Which one should you actually write? For this exercise, Intermediate Approach 1 (strict, typed) is what you should default to — an API key that's empty is never usable, so treat it as missing. Advanced Approach 1 is worth it the moment a caller needs to know *which* key failed, not just read a message. Advanced Approach 2 is worth it once you have several keys where some are genuinely optional-and-empty-is-fine and some aren't — one flexible function beats duplicating `require_env`.

Advanced Approach 3 (Pydantic) is worth reaching for once your config has enough fields that hand-written `if` checks start repeating themselves — don't add a new dependency just for one key.